

PROJECT REPORT

CareSync

Software Production Engineering Major Project

MASTER OF TECHNOLOGY

in

Computer Science & Engineering

Submitted by

Himanshu Shivhare, Mohit Gupta
(MT2024058, MT2024049)



Department of Computer Science and Engineering
International Institute of Information Technology
Bangalore - 560100, India

April 2025

Contents

1	Abstract	3
2	Introduction	4
2.1	Overview	4
2.2	Problem Statement	4
2.3	Objectives	5
2.4	DevOps	5
2.4.1	What is DevOps	5
2.4.2	Benefits of DevOps	6
2.5	Tools and Technologies Used	6
3	Source Code Management	8
3.1	GitHub as SCM	8
3.2	Create GitHub repository	9
4	Docker	10
4.1	Introduction to Docker.....	10
4.2	Docker commands	10
4.3	Dockerfile	11
4.4	Docker-Compose.....	12
5	Ansible	13
5.1	Overview of Ansible	13
5.2	Ansible playbook.....	14
6	Kubernetes	15
6.1	What is Kubernetes	15
6.2	Why Kubernetes	15
6.3	Benefits of Kuberenets	15
6.4	Kubernetes yaml file.....	16
6.5	Minikube	16
6.6	PV and PVC	17
6.7	Horizontal Pod Autoscaler	19
7	Jenkins	20
7.1	Benefits of Jenkins	20
7.2	Jenkins Pipeline.....	21

8 Monitoring	22
8.1 What is Prometheus	22
8.2 What is Grafana	22
9 Conclusion	28
References	29

Chapter 1

Abstract

In the modern digital healthcare ecosystem, efficient, scalable, and secure systems are crucial for delivering high-quality patient care. This project presents a comprehensive implementation of a Healthcare Management System leveraging Microservice Architecture using Spring Boot, integrated with a full-fledged DevOps pipeline. The system is modularized into independent services such as Patient Management, Doctor Service and Appointment Scheduling each deployed as separate Spring Boot microservices. These services communicate via RESTful APIs and are registered through a Service Discovery (Eureka) and routed using an API Gateway.

To ensure continuous delivery and scalability, the project adopts modern DevOps practices including Docker for containerization, Docker Compose and Kubernetes for orchestration, Jenkins and GitHub Actions for CI/CD automation, and Prometheus-Grafana for real-time monitoring and centralized logging. The backend is supported by a PostgreSQL database.

This implementation demonstrates how DevOps combined with microservices can significantly enhance agility, fault tolerance, and maintainability in healthcare applications while ensuring compliance, data integrity, and ease of scaling critical services.

Chapter 2

Introduction

2.1 Overview

The healthcare industry is undergoing a rapid digital transformation, necessitating the development of robust, scalable, and efficient software systems to manage increasing demands for patient care, data handling, and operational efficiency. Traditional monolithic architectures often struggle to keep pace with these evolving requirements, leading to challenges in scalability, maintainability, and deployment.

To address these limitations, microservice architecture has emerged as a preferred paradigm for building complex healthcare applications. By decomposing the system into independent, loosely coupled services, microservices enable better scalability, fault isolation, and continuous deployment. Spring Boot, with its lightweight and production-ready capabilities, is widely used for building such microservices due to its ease of configuration, embedded server support, and seamless integration with modern DevOps tools.

In parallel, DevOps practices have revolutionized software development and operations by automating build, test, deployment, and monitoring processes. The integration of DevOps into microservice-based healthcare systems ensures faster delivery cycles, higher code quality, and enhanced system reliability. This project embodies this synergy by implementing a healthcare management system using Spring Boot microservices and integrating a DevOps pipeline with tools such as Docker, Jenkins, Kubernetes, and Grafana Prometheus.

2.2 Problem Statement

Healthcare systems increasingly struggle with managing core operations like patient registration, appointment booking, and doctor scheduling. Traditional monolithic architectures are inflexible and hard to scale, making even minor updates risky and time-consuming. System-wide downtime and deployment delays are common due to tightly coupled components. CARESYNC tackles these issues with a modular, microservices-based approach tailored for the healthcare domain.

It leverages DevOps practices such as CI/CD pipelines, infrastructure as code (IaC), and automated testing to ensure faster, reliable deployments. Monitoring tools and alerting systems are integrated to enable real-time issue detection and rapid response. CARESYNC supports seamless service scaling during high-traffic periods, improving availability and performance. Decoupled services allow independent updates, reducing downtime and simplifying maintenance.

Security and data integrity are enforced through principles, ensuring compliance with healthcare regulations. Overall, CARESYNC delivers a resilient, scalable, and adaptive solution that embodies the core principles of modern DevOps in a critical healthcare environment.

2.3 Objectives

- To architect and develop a modular Healthcare Management System leveraging Spring Boot-based microservices, with distinct services for patient management, doctor operations, appointment handling, and medical record management.
- To implement service discovery and centralized routing through Eureka and Spring Cloud Gateway for efficient access control and inter-service communication.
- To containerize individual microservices using Docker, ensuring consistent, platform-agnostic deployments and isolated runtime environments.
- To manage and orchestrate containers using Docker Compose or Kubernetes, enabling enhanced scalability, resilience, and simplified service coordination.
- To establish an automated CI/CD pipeline via Jenkins or GitHub Actions, promoting fast, consistent, and error-free delivery of code changes.
- To incorporate centralized monitoring and logging with Prometheus and Grafana, providing real-time visibility into system performance and facilitating proactive troubleshooting.
- To highlight the value of DevOps practices in the healthcare domain, including faster deployment cycles, improved system reliability, and reduced recovery time from failures.

2.4 DevOps

2.4.1 What is DevOps

DevOps is a modern software development approach that combines "Development" (Dev) and "Operations" (Ops) to streamline and automate the entire software delivery lifecycle. It emphasizes close collaboration between software developers and IT operations teams, aiming to shorten development cycles, increase deployment frequency, and deliver high-quality software consistently.

Unlike traditional development models where development and operations work in silos, DevOps fosters a culture of shared responsibility, continuous feedback, and automation. It involves practices like Continuous Integration (CI), Continuous Delivery/Deployment (CD), Infrastructure as Code (IaC), and automated monitoring, all aimed at making software delivery faster, more efficient, and more reliable.

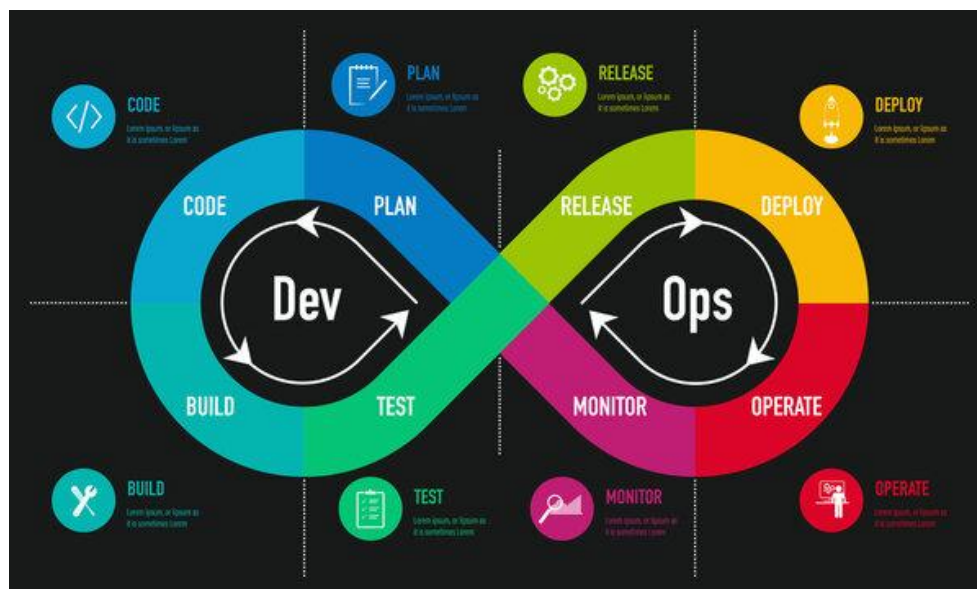


Figure 2.1: DevOps

2.4.2 Benefits of DevOps

- **Accelerated Delivery:** Automated CI/CD workflows streamline the development lifecycle, enabling faster rollout of new features and updates.
- **Higher Deployment Frequency:** Small, incremental releases reduce risk and simplify issue tracking and rollback processes.
- **Cross-Team Collaboration:** DevOps fosters seamless integration between development, quality assurance, and operations, enhancing overall productivity.
- **Greater System Reliability:** Real-time monitoring and alert systems help maintain high availability and ensure rapid response to incidents.
- **Effortless Scalability:** Container orchestration tools allow applications to scale efficiently based on demand.
- **Lower Downtime and Failure Rates:** Robust testing and observability practices minimize production issues and accelerate recovery during failures.

2.5 Tools and Technologies Used

- Programming Language: Java/Spring Boot
- Microservice Architecture
- Version Control: Git and GitHub
- Build Automation: Maven
- CI/CD Pipeline: Jenkins
- Configuration as Code: Ansible
- Containerization: Docker
- Orchestration Tool: Kubernetes
- Monitoring Tool : Grafana/Prometheus

Chapter 3

Source Code Management

3.1 GitHub as SCM

Source Code Management (SCM) plays a vital role in the software development lifecycle by tracking and managing changes to the codebase over time. It enables teams to collaborate efficiently, maintain a complete history of changes, and revert to previous versions when necessary. By integrating SCM with DevOps practices, teams can automate development workflows, trigger CI/CD pipelines, and enforce code quality standards directly through the version control system.

In this project, **GitHub** serves as the primary SCM tool. Built on top of Git—the most widely used distributed version control system—GitHub offers a powerful and intuitive web-based platform for managing repositories, branches, pull requests, and collaborative coding. Its integration capabilities make it an ideal choice for supporting automated deployments and team-based development within a DevOps ecosystem.

Key features of GitHub:

- **Version Control:** Tracks all changes made to the codebase with commit history and logs.
- **Branching and Merging:** Enables feature branching and safe integration through pull requests.
- **Collaboration:** Allows multiple developers to contribute to the same repository with controlled access.
- **CI/CD Integration:** Triggers automated builds and deployments using GitHub Actions or external tools like Jenkins.
- **Issue Tracking:** Facilitates agile project management with issue tracking and project boards.
- **Code Review:** Ensures code quality through peer reviews and approval workflows.

3.2 Create GitHub repository

The series of commands are executed to push local repository to the remoter repository

- git init
- git status
- git add
- git commit -m "Commit Message"
- git remote add origin https://github.com/him15/HealthCare_FullProject.git
- git push -u origin main

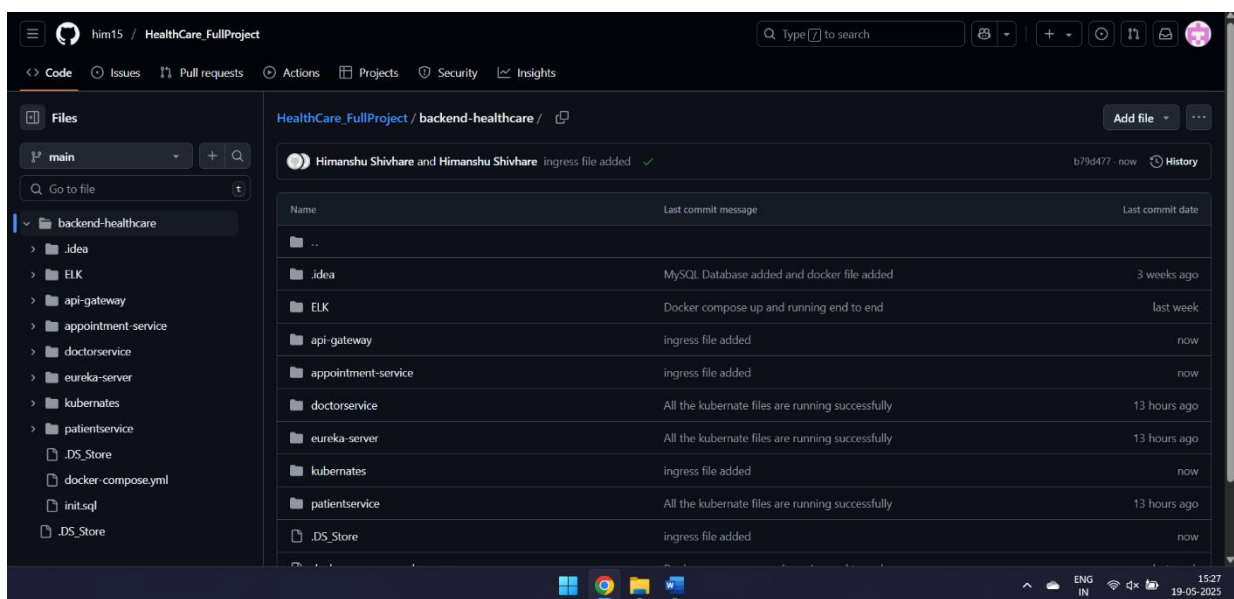


Figure 3.1: GitHub Repo

GitHub Repo: https://github.com/him15/HealthCare_FullProject.git

Chapter 4

Docker

4.1 Introduction to Docker

Docker is an open-source containerization platform that allows developers to package applications along with all necessary dependencies, libraries, and runtime into lightweight, portable containers. These containers provide isolated environments, ensuring that applications run consistently across various stage from development and testing to production regardless of the underlying infrastructure.

Unlike traditional virtual machines, Docker containers share the host system's kernel, making them significantly more lightweight, faster to start, and resource-efficient. This approach resolves the common “it works on my machine” issue by encapsulating the complete runtime environment within each container.

In this healthcare microservices project, Docker is utilized to containerize each individual Spring Boot service, including key components such as:

- appointment-service
- config-server
- doctor-service
- eureka-server
- gateway
- postgresSQL
- pgAdmin

4.2 Docker commands

We can build the docker image using the following command:

⇒ `docker build -t USERNAME/IMAGE NAME: TAG`

To push the docker image use command:

⇒ `docker push USERNAME/REPOSITORY NAME:tagname`

The docker image is pushed from jenkins after the build stage and pulled on from ansible job. The docker image can be run using command

`docker run -i -t image_name.`

After pushing the created docker image to Docker Hub, we can check on our Docker hub profile.

4.3 Dockerfile

A **Dockerfile** is a simple text file that contains a series of instructions used by Docker to automatically build a Docker image. It specifies all the components required to set up the application environment such as the base image, system libraries, dependencies, and commands to install and run the application.

Using Dockerfiles ensures that the application environment is **portable, consistent, and reproducible** across development, testing, and production stages. Instead of performing environment setup manually, developers can define the process once in a Dockerfile and reuse it across any platform or machine.

Below is the Dockerfile for the **Patient Service**. Similar Dockerfiles have been implemented for the **Doctor Service** and **Appointment Service**, following the same structure and best practices.

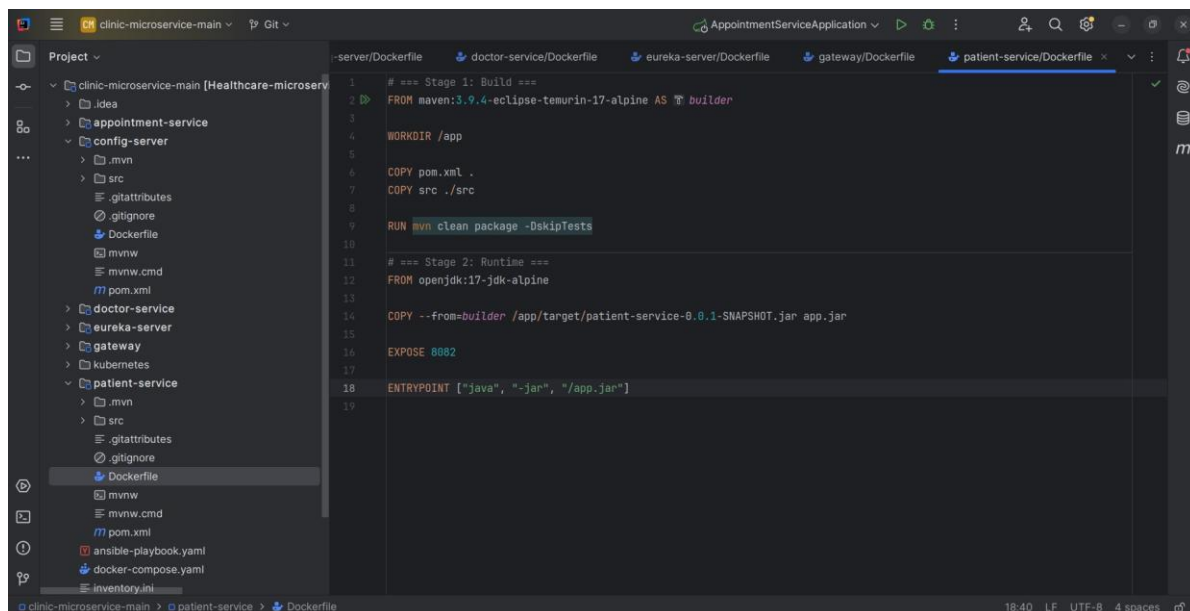
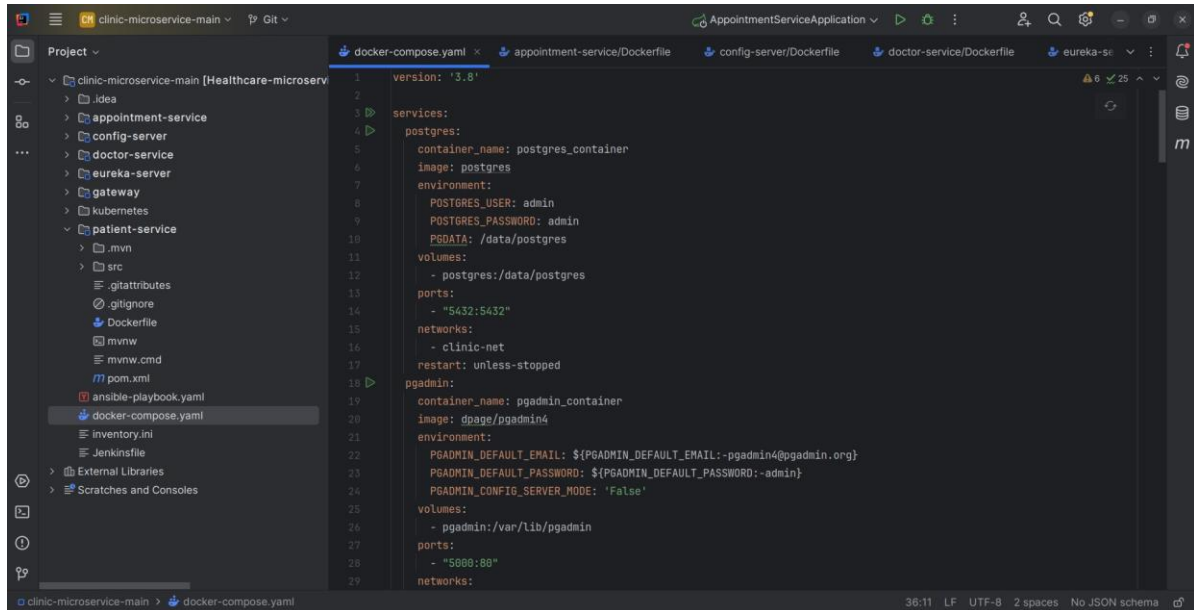


Figure 4.1: Patient dockerfile

To build the docker image and push it to Docker Hub remote repository, we will specify steps in Jenkins pipeline script.

The Docker image is built and pushed on the Docker Hub repository of the user. The local image on the machine is deleted after the image is pushed on the Docker Hub.

4.4 Docker-Compose



Docker Compose is a powerful tool that simplifies the definition and management of multi-container Docker applications through a single configuration file, usually named `docker-compose.yml`. It allows developers to define the services, networks, and volumes required for the entire application stack in one place.

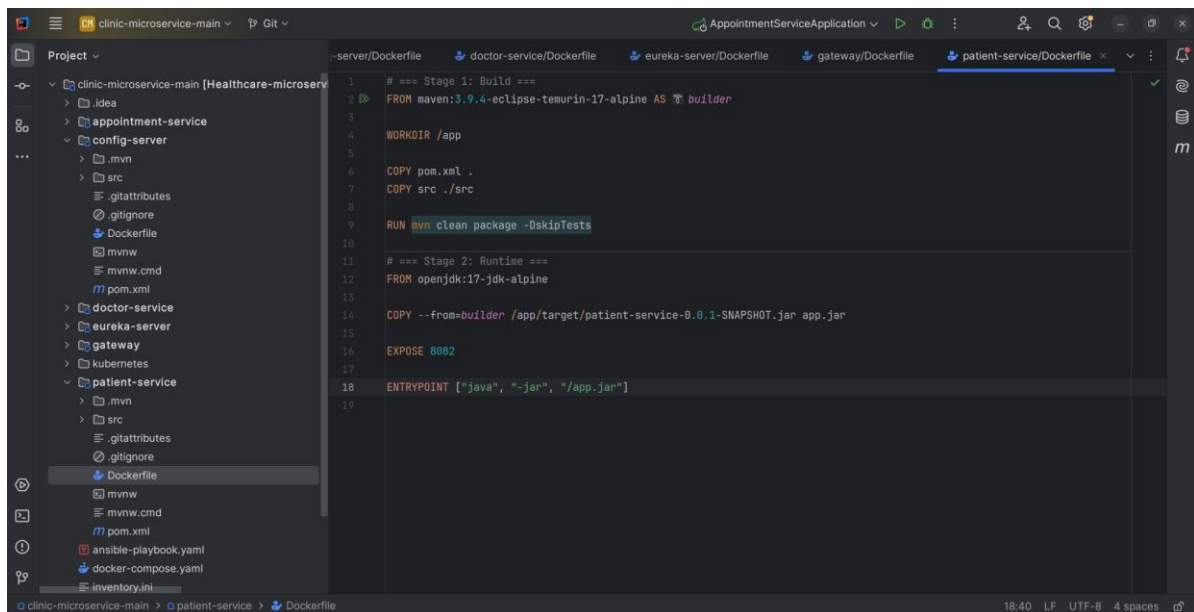


Figure 4.2: Docker-Compose

Chapter 5

Ansible

5.1 Overview of Ansible

Ansible is an open-source automation tool widely used for configuration management, application deployment, and orchestration of IT infrastructure. It enables DevOps teams to automate routine tasks and manage complex systems with consistency and efficiency.

Ansible operates using simple, human-readable automation scripts called **playbooks**, written in YAML. This approach lowers the barrier to entry, making it accessible even for those with limited scripting experience. One of Ansible's standout advantages is its **agentless architecture** it communicates with remote systems over SSH, eliminating the need to install additional software on managed nodes.

Key Features of Ansible:

- **Agentless Execution:** No agents required on target machines, simplifying maintenance.
- **Human-Readable Syntax:** Automation tasks are written in easy-to-understand YAML files.
- **Idempotent Operations:** Tasks only apply changes when needed, avoiding duplication.
- **Highly Scalable:** Capable of managing thousands of servers and containers concurrently.
- **Modular Design:** Encourages reusable automation through roles, tasks, and variables.

5.2 Ansible playbook

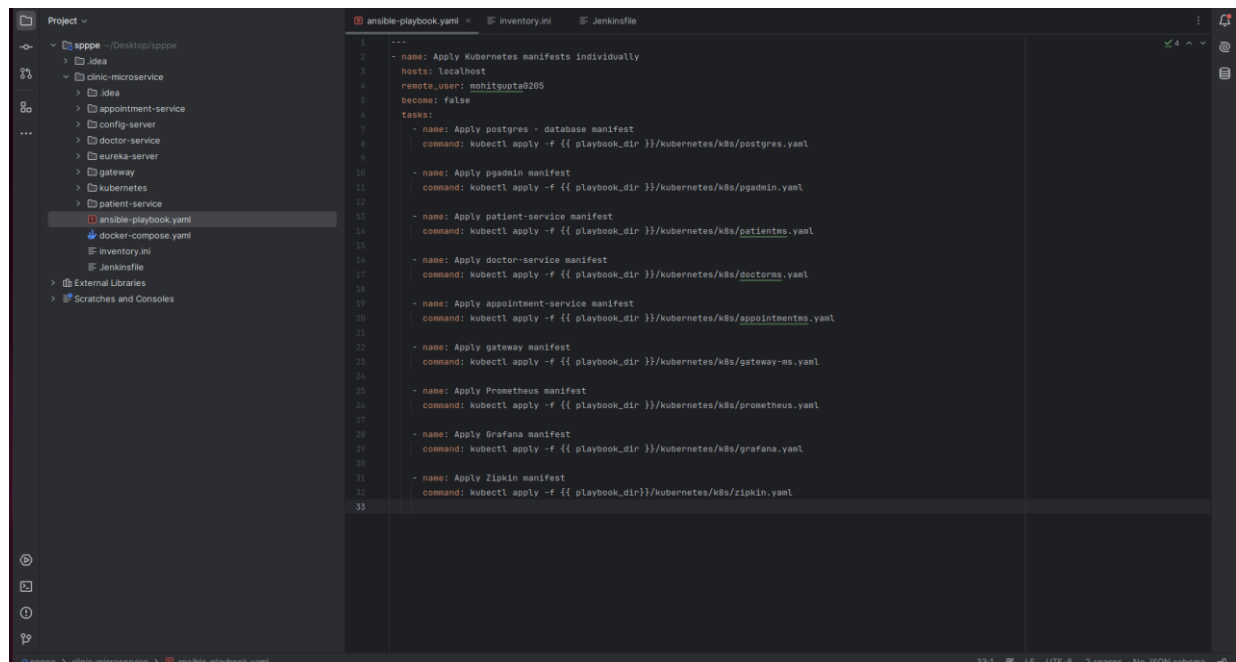


Figure 5.1: Ansible-playbook

This **Ansible playbook** is crafted to automate the deployment of Kubernetes resources for a microservices-based healthcare application. Executed locally, it utilizes the `kubectl` command to apply a series of Kubernetes manifest files in a streamlined and repeatable manner.

Chapter 6

Kubernetes

6.1 What is Kubernetes

Kubernetes (commonly referred to as K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. Initially developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF), Kubernetes has become the de facto standard for managing containers in production environments.

While Docker is great for containerizing applications, it doesn't provide built-in tools to manage containers in a distributed, scalable, and fault-tolerant way. This is where Kubernetes comes in.

- Deploy and scale containers automatically
- Ensure high availability and fault tolerance
- Roll out updates without downtime
- Monitor and self-heal applications
- Manage services, networking, and storage dynamically

6.2 Benefits of Kubernetes

- **Automated Scaling:** Automatically increases or decreases the number of running containers based on CPU/memory usage.
- **Self-Healing:** Detects failed containers and restarts, replaces, or reschedules them automatically.

- **Rolling Updates & Rollbacks:** Safely deploys application changes with zero downtime and allows easy rollback if needed.
- **Service Discovery & Load Balancing:** Automatically exposes containers and balances traffic across healthy instances.
- **Resource Optimization:** Efficiently allocates hardware resources to maximize utilization.
- **Platform Agnostic:** Can be deployed on-premises, in the cloud, or in hybrid environments.
- **Infrastructure as Code (IaC):** Declarative configuration through YAML manifests makes the system easy to reproduce and maintain.

6.3 Kubernetes yaml file

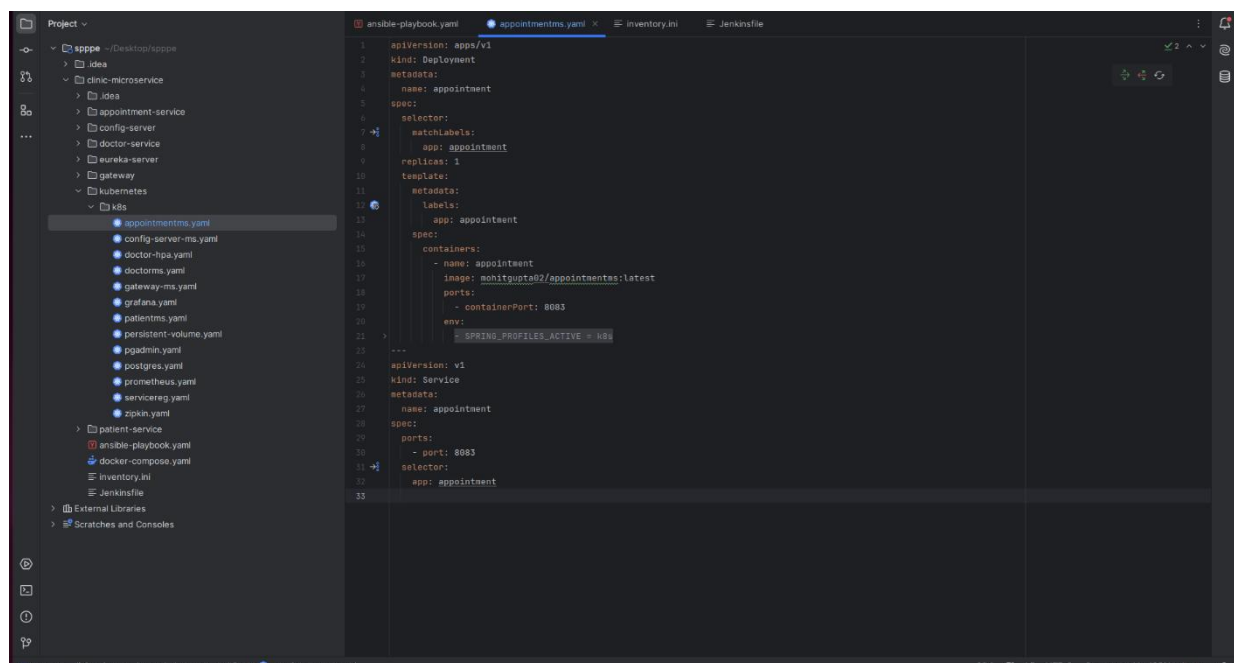


Figure 6.1: Kubernetes-yaml file

Above specified the appointment kubernetes yaml file as like this file we also created several kubernetes files of patient, doctor, postgres, pgadmin, grafana, prometheus etc. for Orchestrating between multiple docker containers

6.4 Minikube

Minikube is a lightweight tool that enables developers to run a local Kubernetes cluster on their personal machines. It is primarily designed for **development, testing, and learning**, offering an easy way to explore Kubernetes features without requiring a full cloud infrastructure.

Minikube provisions a **single-node Kubernetes cluster** inside a virtual machine or container, allowing you to work with familiar tools like kubectl. This setup is ideal for prototyping and

debugging microservices-based applications in an environment that closely mimics real-world Kubernetes deployments—making it a valuable asset before moving to production.

6.5 PV and PVC

A PersistentVolume (PV) is a piece of storage in a Kubernetes cluster that has been provisioned by an administrator or dynamically provisioned using StorageClasses. It is independent of any specific pod, meaning the data stored there can persist even if the pod is deleted.

Think of a PV as a physical or virtual disk inside the cluster.

A PersistentVolumeClaim (PVC) is a request for storage by a user or application. It specifies how much storage is needed and with what access mode (e.g., read/write).

When a PVC is created, Kubernetes matches it with an available PV that satisfies the request.

6.6 Horizontal Pod Autoscaler

Horizontal Pod Autoscaler (HPA) is a Kubernetes resource that automatically adjusts the number of running pods in a Deployment, ReplicaSet, or StatefulSet based on real-time metrics such as CPU usage, memory consumption, or custom-defined metrics.

Why Use HPA?

Improves Performance: Automatically scales up pods during high load to maintain responsiveness.

Optimizes Resource Usage: Scales down pods during low traffic periods to reduce resource consumption.

Ensures High Availability: Maintains service continuity and responsiveness without manual intervention.

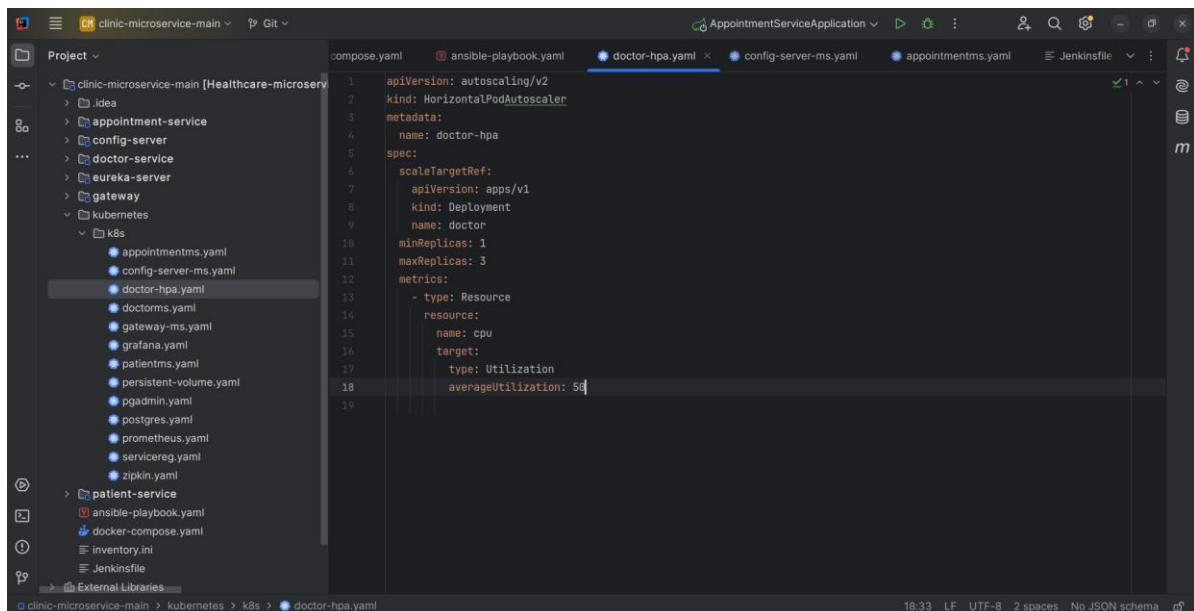


Figure 6.5: HPA applied to doctor-microservice

Chapter 7

Jenkins

Jenkins is an open-source automation server widely used to implement Continuous Integration (CI) and Continuous Delivery (CD) pipelines. It automates the process of building, testing, and deploying software, enabling development teams to deliver updates to users faster and more reliably.

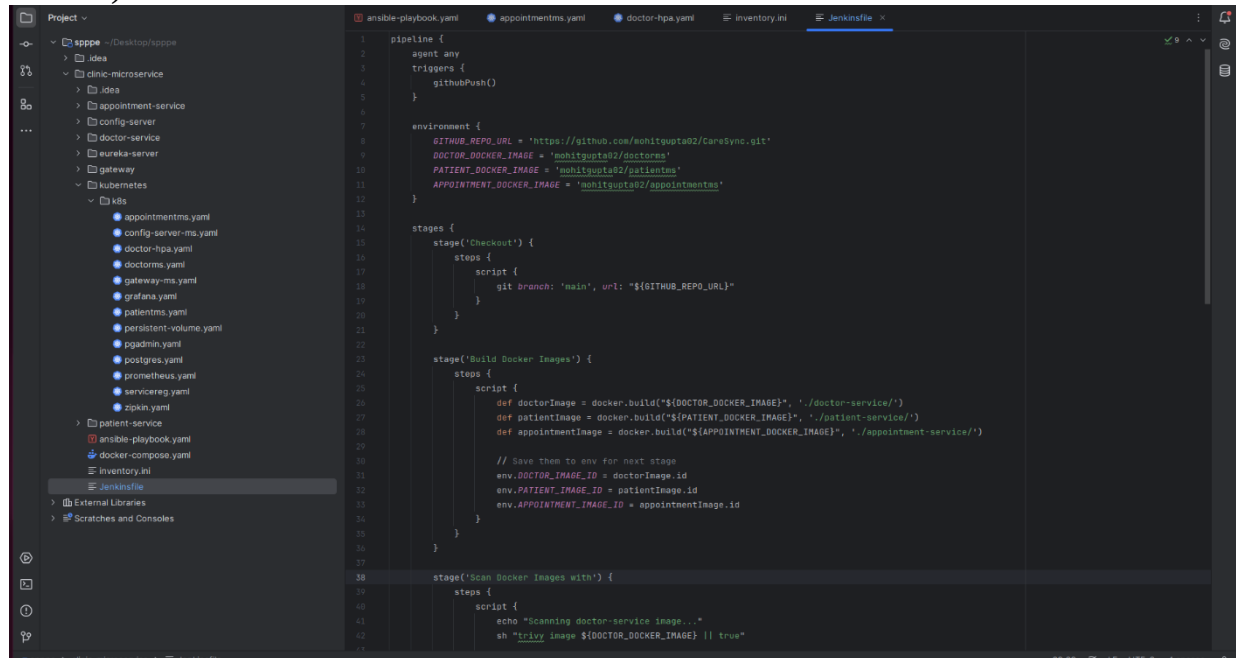
Jenkins supports hundreds of plugins that integrate with almost every tool in the DevOps ecosystem, making it highly extensible and customizable. It provides a web-based interface to configure pipelines, monitor build status, and manage jobs, which helps streamline and standardize software delivery workflows.

7.1 Benefits of Jenkins

- **Improved Developer Productivity:** Automation reduces manual tasks, freeing developers to focus on coding.
- **Faster Release Cycles:** Enables continuous integration and delivery, accelerating the feedback loop.
- **Flexibility:** Supports various programming languages, build tools, and deployment platforms.
- **Extensibility:** Large ecosystem of plugins for integration with SCM tools (like GitHub), Docker, Kubernetes, and testing frameworks.
- **Open Source:** Free to use and backed by a strong community, ensuring regular updates and improvements.

7.2 Jenkins Pipeline

A Jenkinsfile is a text file that contains the definition of a Jenkins pipeline. It is written using Groovy-based syntax and stored in the root of your source code repository (e.g., GitHub).



```
1 pipeline {
2   agent any
3   triggers {
4     githubPush()
5   }
6
7   environment {
8     GITHUB_REPO_URL = 'https://github.com/mohitgupta02/CareSync.git'
9     DOCTOR_DOCKER_IMAGE = 'mohitgupta02/doctores'
10    PATIENT_DOCKER_IMAGE = 'mohitgupta02/patientms'
11    APPOINTMENT_DOCKER_IMAGE = 'mohitgupta02/appointmentms'
12  }
13
14  stages {
15    stage('Checkout') {
16      steps {
17        script {
18          git branch: 'main', url: "${GITHUB_REPO_URL}"
19        }
20      }
21    }
22
23    stage('Build Docker Images') {
24      steps {
25        script {
26          def doctorImage = docker.build("${DOCTOR_DOCKER_IMAGE}", './doctor-service/')
27          def patientImage = docker.build("${PATIENT_DOCKER_IMAGE}", './patient-service/')
28          def appointmentImage = docker.build("${APPOINTMENT_DOCKER_IMAGE}", './appointment-service/')
29
30          // Save them to env for next stage
31          env.DOCTOR_IMAGE_ID = doctorImage.id
32          env.PATIENT_IMAGE_ID = patientImage.id
33          env.APPOINTMENT_IMAGE_ID = appointmentImage.id
34        }
35      }
36    }
37
38    stage('Scan Docker Images with') {
39      steps {
40        script {
41          echo "Scanning doctor-service image..."
42          sh "trivy image ${DOCTOR_DOCKER_IMAGE} --exit-code 0"
43        }
44      }
45    }
46  }
47}
```

Figure 7.1: Jenkinsfile

Chapter 8

Monitoring

8.1 What is Prometheus

Prometheus is an open-source monitoring and alerting system built for recording real-time metrics in a highly efficient time-series database. It is extensively used in cloud-native environments, especially within Kubernetes ecosystems.

Prometheus collects metrics from configured endpoints, stores them reliably, and offers a powerful query language called **PromQL** for flexible data analysis. It is ideal for monitoring system performance, tracking resource usage such as CPU and memory, measuring service uptime, and gathering custom application metrics helping teams maintain the health and reliability of their infrastructure.

8.2 What is Grafana

Grafana is an open-source analytics and visualization platform that integrates smoothly with Prometheus. It enables the creation of interactive, customizable dashboards to display real-time metrics through charts, graphs, and alerts.

Grafana empowers developers and DevOps teams to gain valuable insights into system performance, quickly identify potential issues, and support informed, data-driven decision-making. Together, **Prometheus** handles data collection, while **Grafana** provides intuitive visualization forming a powerful monitoring solution for modern distributed systems like Kubernetes-based microservices.

Chapter 9

Conclusion

This project demonstrates the successful development and deployment of a Healthcare Management System built with Spring Boot microservices and supported by modern DevOps practices. Docker and Kubernetes handle containerization and orchestration, while Jenkins automates the CI/CD pipeline for rapid, reliable releases. Real-time monitoring is enabled through Prometheus and Grafana, and Zipkin provides distributed tracing to enhance observability. GitHub serves as the source code management platform, streamlining collaboration and version control. Together, these technologies create a scalable, efficient, and maintainable healthcare solution that exemplifies the power of combining microservices architecture with DevOps.

References

- Docker Documentation: <https://docs.docker.com/>
- Kubernetes Overview: <https://kubernetes.io/docs/concepts/overview/>
- Jenkins Learning Resources: <https://codefresh.io/learn/jenkins/>
- Ansible Documentation: <https://devdocs.io/ansible/>
- Prometheus and Grafana Visualization Guide:

<https://prometheus.io/docs/visualization/grafana/>
- Kubernetes Horizontal Pod Autoscaler (HPA):

<https://spacelift.io/blog/kubernetes-hpa-horizontal-pod-autoscaler>
- Docker Compose Hub: <https://hub.docker.com/r/docker/compose>